

```
void kobject_del(struct kobject *kobj);
```

This interface will delete the directory of *sysfs* represented by the kobject pointer passed to it.

For example, to delete the *example_dev* directory that was created as an example, the following function call is used:

```
kobject_del(example_dev_kobj);
```

Sysfs directory renaming

In case there is a need to rename the directory created in *sysfs*, the following functions are available:

```
int sysfs_rename_dir(struct kobject *kobj, const char *new_name)
```

...where, *kobj* is the kobject pointer corresponding to the directory that has to be renamed, and

new_name is to be given to the directory. Since the *sysfs* interface is normally not directly called, the following kobject interface is called for the renaming purpose:

```
int kobject_rename(struct kobject *kobj, const char *new_name)
```

...where,

kobject: This is the kobject pointer of the directory to be renamed.

new_name: This is the new name to be given to the directory.

Kobject reference counting

Each kobject has a member *struct kref*, which is a reference counter. Whenever any module wishes to use the kobject already created, this reference count is incremented, and whenever that module wishes to stop using kobject, the reference count is decremented to maintain the number of modules using the kobject or the *sysfs* directory represented by the kobject. Whenever the reference count reaches 0, the memory related to the kobject which was allotted by *kobject_create_and_add* is released, and whenever a kobject is created using *kobject_create_and_add*, it internally initialises the *kref* reference counter by one. This indicates the kobject is being used by the current module, and then *kobject_create_and_add* internally creates the directory for the kobject. Whenever any new module wishes to use the directory represented by that kobject, the reference counter has to be incremented.

The reference count is maintained in the kobject using the following functions:

```
struct kobject* kobject_get(struct kobject *kobj);
```

```
struct kobject* kobject_put(struct kobject *kobj);
```

kobject_get increments the reference count of the kobject passed to it, whereas *kobject_put* decrements the reference count of the kobject passed to it. So while writing drivers or any kernel module, whenever any *kobject* of the other module is directly used, *kobject_get* should be called to increment the reference count of the kobject. When the use of the kobject is finished, *kobject_put* should be called to decrement its reference count.

Whenever *kobject_put* is called, passing the kobject pointer to it, it decrements the reference count. When the reference count reaches zero, it frees the memory of the kobject pointer, and the kobject pointer can no longer be used.

Files in sysfs

In the above example, we have created the *example_dev* directory in the */sys/kernel* folder. Suppose the example had some attribute called *example_info* which we want to expose to the user space using *sysfs*, then the feature used is called *sysfs* attributes.

Attributes are represented as regular files in *sysfs* with one value per file. Attributes are of the type *struct attribute* (defined in *include/linux/sysfs.h*). *Struct attribute* has the following two important pieces of information.

name: This is the name of the attribute(file) to be created.

mode: This is the permission with which the file is to be created. A simple attribute has no means by which it can be read or written; it needs wrapper routines for reading and writing. For this purpose, kobject defines a special structure called *struct kobj_attribute* (defined in *include/linux/kobject.h*) as follows:

```
struct kobj_attribute {
```

```
struct attribute attr;
```

```
    ssize_t (*show)(struct kobject *kobj, struct kobj_attribute *attr, char *buf);
```

```
    ssize_t (*store)(struct kobject *kobj, struct kobj_attribute *attr, const char *buf, size_t count);
};
```

...where, *attr* is the attribute representing the file to be created, *show* is the pointer to the function that will be called when the file is read in *sysfs*, and *store* is the pointer to the function which will be called when the file is written in *sysfs*.

Suppose we need to create our *example_info* file in the */sysfs/kernel/example_dev* directory that was created as an example; then the following method is the way to define the attribute:

```
struct kobj_attribute example_info_attr =
__ATTR(example_info, 0666, example_show, example_store);
```

...where *__ATTR* is the macro, as shown below: